

get_operating_mode

REFERENCE

API consumer

read back the reader's current RFID [operating-mode](#) configuration

Overview

`get_operating_mode` reads back the reader's current RFID operating-mode configuration — the active profile, radio start/stop conditions, Gen2 `query` parameters, select pre-filters, armed access operations, and tag-metadata reporting flags — under `operatingMode.operatingModes`. It is read-only. It is a **control-plane (CTRL) command** — configure the mode with `set_operating_mode`.

This is a **CTRL-endpoint command**: it is published on the **control** endpoint (`<tenantId>/CTRL/clients/cmnd/<serial>`) and answered only there (not on the [MGMT/MDM](#) channel). Returning a response requires an **active CTRL endpoint** (configured with `config_endpoint`, `epType: CTRL`, `activate: true`). See [Topic Mapping](#).

Details

Property	Value
Interaction Pattern	Request / Response (app publishes a command; device publishes the correlated reply)
State Change	READ-ONLY
Required Request Fields	<code>command</code> , <code>requestId</code> (no other payload)
Applies To	Endpoint: Control (CTRL). Reader: RFD40 series, RFD90 series
API Version	<code>v1.1</code> (API Version Reporting)
Related	<code>set_operating_mode</code> , <code>control_operation</code> , <code>config_endpoint</code> , <code>reboot</code>

When to Use

Use `get_operating_mode` to inspect or audit the reader's RFID configuration:

- Before an inventory** — confirm the profile, `query` parameters, and trigger conditions are what you expect.
- After `set_operating_mode`** — verify the mode was applied (there is no configuration echo on the set command's response).

- **After a reboot** — the operating mode reverts to default on reboot (see the volatility note); read it back to confirm the current state.
- **Check reporting** — see which `tagMetadataToEnable` fields the reader will include on tag reads.

Topic Mapping

Message	Direction	Topic	Note
Request	App → Reader	<code><tenantId>/CTRL/clients/cmnd/<serial></code>	On the control endpoint. Requires an active CTRL endpoint .
Response	Reader → App	<code><tenantId>/CTRL/clients/resp/<serial></code>	Correlated reply; the in-payload <code>requestId</code> links the twin topics.

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

Unlike the MGMT commands, control commands are answered **only on the CTRL endpoint** — publishing `get_operating_mode` to the MGMT/MDM command topic produces no reply.

If no CTRL endpoint is active, add or activate one with `config_endpoint` (`epType: CTRL`, `activate: true`) before sending control commands. The `requestId` you send is echoed in the response.

See [Topics and Channels](#).

Key Fields

- `operatingMode.operatingModes.profiles` (response) — the active profile (e.g. `BALANCED_PERFORMANCE`); **ADVANCED** surfaces `advancedConfigurations`.
- `query` (response) — the Gen2 session / inventory-state / SL-flag / tag-population parameters driving inventory.
- `radioStartConditions` / `radioStopConditions` (response) — the trigger semantics for starting/stopping inventory.
- `tagMetadataToEnable` (response) — which metadata fields (RSSI, EPC, TID, ...) the reader reports on each tag read.
- `response.code` (response) — read this to determine success vs failure. The contract requires it; note the reported success-path omission. See [Response Codes](#).

Request Payload

The request is a minimal envelope — just `command` (pinned to `get_operating_mode`) and `requestId`. There are no parameters.

Request payload

```
1 {
2   "command" : "get_operating_mode" ,
3   "requestId" : "abc123"
4 }
```

Request payload

* required

— not applicable

Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMPLE	DESCRIPTION
1	command *	enum	get_operating_mode	get_operating_mode	Fixed command name (pinned by the schema enum).
2	requestId *	string	—	abc123	A unique identifier for the request; echoed back in the response. There is no other payload.

Response Payload

On success the device returns the envelope (`command` , `requestId` , `apiVersion` , `response`) with the current mode under `operatingMode.operatingModes` . The device returns only the configured sub-objects — `advancedConfigurations` , `accessOperations` , and `select` appear only when configured. The **Example** tab shows a default `BALANCED_PERFORMANCE` read-back.

Response payload

```
1  {
2    "command": "get_operating_mode",
3    "requestId": "abc123",
4    "apiVersion": "V1.1",
5    "operatingMode": {
6      "operatingModes": {
7        "profiles": "BALANCED_PERFORMANCE",
8        "inventoryProtocol": "GEN2",
9        "radioStartConditions": {
10         "trigger": "IMMEDIATE",
11         "startDelay": 0,
12         "repeat": false
13       },
14       "radioStopConditions": {
15         "trigger": "IMMEDIATE"
16       },
17       "query": {
18         "session": "SESSION_1",
19         "inventoryState": "STATE_A",
20         "slFlag": "ALL",
21         "tagPopulation": 30
22       },
23       "tagMetaDataToEnable": {
24         "RSSI": true,
25         "PHASE": false,
26         "SEENCOUNT": true,
27         "CHANNEL": false,
28         "PC": false,
29         "XPC": false,
30         "CRC": false,
31         "EPC": false,
32         "TID": false,
33         "USER": false,
34         "MAC": false,
35         "HOSTNAME": false
36       }
37     },
38   },
39   "response": {
40     "code": 0,
41     "description": "Success"
42   }
43 }
```

Response payload

* required • — not applicable • Colored badge = JSON type

#	FIELD	TYPE	ENUM / UNITS	EXAMP
1	command *	string	—	get_c
2	requestId *	string	—	abc12
3	apiVersion *	enum	V1.0 V1.1 V1.2 V1.21 V1.22	V1.1
4	operatingMo de	object	—	—
5	operating Modes	object	—	—
6	profil es	enum	FAST_READ CYCLE_COUNT DENSE_READERS OPTIMAL_BATTERY BALANCED_PERFORMANCE ADVANCED	BALAN
7	advanc edConf igurat ions	object	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAMP
8	transmitPower	number	0-300	270
9	linkProfile	enum	M4_256K M2_240K M2_256K M2_320K M4_240K M4_320K FM0_0K FM0_320K M8_240K M8_256K M8_320K	M2_24
10	session	enum	SESSION_0 SESSION_1 SESSION_2 SESSION_3	SESSI
11	dynamicPower	boolean	—	true
12	inventoryProtocol	enum	GEN2 GEN2X HYBRID	GEN2
13	accessOperations	array	—	—
14	operationType	enum	READ WRITE ACCESS LOCK KILL	READ
15	config	object	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAMP
16	m e m o r y B a n k	enum	EPC TID USER RESERVED	EPC
17	o f f s e t	integer	—	1
18	l e n g t h	integer	—	2
19	d a t a	string	—	FFFFE
20	p a s s w o r d	string	—	0000C

#	FIELD	TYPE	ENUM / UNITS	EXAMP
21	lockMemBank	enum	EPC ACCESS_PWD KILL_PWD TID USER ALL	TID
22	lockAccession	enum	READ_AND_WRITE PERMANENT_LOCK PERMANENT_UNLOCK UNLOCK	UNLOCK
23	radioStartConditions	object	—	—
24	trigger	enum	PRESSED RELEASED IMMEDIATE	IMMEDIATE
25	startDelay	integer	—	3000
26	repeat	boolean	—	false
27	radioTopConditions	object	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAMP
28	trigger	enum	PRESSED RELEASED IMMEDIATE	IMMEI
29	tag Count	integer	—	10
30	stopTimeOut	integer	—	10000
31	inventory Count	integer	—	0
32	query	object	—	—
33	session	enum	SESSION_0 SESSION_1 SESSION_2 SESSION_3	SESSI
34	inventory State	enum	STATE_A STATE_B STATE_AB	STATE
35	slF lag	enum	ALL ASSERTED DEASSERTED	ALL
36	tag Population	integer	min 0	30
37	select	array	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAMP
38	enable *	boolean	—	true
39	tagParameter *	string	—	2222E
40	memoryBank *	enum	EPC TID USER	EPC
41	offset *	integer	—	32
42	action *	enum	INV_A_NOT_INV_B_OR_ASRT_SL_NOT_DSRT_SL INV_A_OR_ASRT_SL NOT_INV_B_OR_NOT_DSRT_SL INV_A2BB2A_NOT_INV_A_OR_NEG_SL_NOT_ASRT_SL INV_B_NOT_INV_A_OR_DSRT_SL_NOT_ASRT_SL INV_B_OR_DSRT_SL NOT_INV_A_OR_NOT_ASRT_SL NOT_INV_A2BB2A_OR_NOT_NEG_SL	INV_E
43	target *	enum	SESSION_SL SESSION_0 SESSION_1 SESSION_2 SESSION_3	SESSI
44	length *	integer	—	16
45	tagMetadataEnable	object	—	—

#	FIELD	TYPE	ENUM / UNITS	EXAMP
46	RSSI	boolean	—	true
47	PHASE	boolean	—	false
48	SEENCOUNT	boolean	—	true
49	CHANNEL	boolean	—	false
50	PC	boolean	—	false
51	XPC	boolean	—	false
52	CRC	boolean	—	false
53	EPC	boolean	—	false
54	TID	boolean	—	false
55	USER	boolean	—	false
56	MAC	boolean	—	false
57	HOSTNAME	boolean	—	false
58	response *	object	—	—
59	code *	integer	0-101	0
60	description *	string	—	Success

The contract requires `response.code`. Readers have been reported to omit it on a successful reply, returning just `response: { "description": "Success" }` — treat that as success too. See [Success without a code](#).

As reported, a successful read can arrive with `code` omitted — the same envelope and `operatingMode` payload. This form does not validate against the contract; the reply with `code: 0` above is the documented form. The mode object is abridged here:

Response — success reported without code (not accepted by the contract; operatingMode abridged)

```
1  {
2    "command" : "get_operating_mode" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.1" ,
5    "operatingMode" : {
6      "operatingModes" : {
7        "profiles" : "BALANCED_PERFORMANCE"
8      }
9    },
10   "response" : {
11     "description" : "Success"
12   }
13 }
```

Response Semantics

- **The read-back masks access passwords.** When access operations are armed, `get_operating_mode` returns their structure but echoes `config.password` as empty (`"`) — the stored credential is never returned. The real password is carried only in the `set operating mode` request (mask it there). A default read-back has no `accessOperations` at all.
- **FAST_READ is enumerated but not supported.** The `profiles` enum lists `FAST_READ`, but it is **currently not supported** by the firmware (per the schema's own description). Use `CYCLE_COUNT`, `DENSE_READERS`, `OPTIMAL_BATTERY`, `BALANCED_PERFORMANCE`, or `ADVANCED`.
- **Conditional sub-objects.** `advancedConfigurations` appears only when `profiles` is `ADVANCED`; `accessOperations` and `select` appear only when configured. A default read-back typically returns just `profiles`, `inventoryProtocol`, `radioStartConditions`, `radioStopConditions`, `query`, and `tagMetadataToEnable`.

Response Codes

`get_operating_mode` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

Code	Description
0	Success — the operating mode was retrieved.
3	Not able to retrieve information (<code>IOT_ERROR_INFO_NOT_AVAILABLE</code>) — the device could not gather the operating mode at this time. Retry after a short delay; if persistent, reboot.
101	Default error — generic catch-all.

The contract requires `response.code`; readers have been reported to omit it on success — see [Success without a code](#).

Examples

Publish `get_operating_mode` over the CTRL endpoint (with an active CTRL endpoint); the device replies with the snapshot shown under [Response Payload](#). There are no request variants — the command takes no parameters.

Notes and Caveats

- **Operating mode is volatile.** The operating mode resets to default (`BALANCED_PERFORMANCE` , armed `accessOperations` cleared) on `reboot` . Read it back after a reset to confirm the current state; re-apply with `set_operating_mode` if you were relying on a non-default configuration.

Read `get_operating_mode` to confirm the reader's RF configuration before running an inventory with `control_operation` , and to verify a change applied by `set_operating_mode` . For the concept-level view, see [Operating Mode](#).

Related:  [set_operating_mode](#) ·  [control_operation](#) ·  [config_endpoint](#) ·  [reboot](#) ·  [Response Codes](#)